AI in Fluids – Week 2

Neural Networks for Fluid Surrogates and Rarefied-Flow Variables

Instructor: Ehsan Roohi
M&I-ENG 690A: AI in Fluid Mechanics

Summer Online Course

Course theme: learn the physics, learn the code, learn from data, and test the limits.
Week 2 theme: understand what a neural network is, how it makes a prediction, how it learns from data, and how this learning workflow becomes a small rarefied-flow surrogate.

Contents

1 Purpose of this lecture **3**

2 From a CFD field to a machine-learning dataset **4**

3 Supervised learning: vocabulary and workflow **5**
3.1 Regression and classification 5

4 A first analogy: a linear decision rule **6**

5 The artificial neuron **6**
5.1 Physical interpretation of weights and bias 7

6 Activation functions **7**
6.1 Derivatives of common activations 8

7 Targets, predictions, errors, and loss functions **8**

8 Learning: how one neuron changes its weights **9**
8.1 Worked numerical example 10

9 Learning rate, instability, and slow learning **11**

10 From one neuron to a multi-layer perceptron **11**
10.1 Why hidden layers help 12

11 Backpropagation: how deep networks learn **12**
11.1 Chain rule idea 12
11.2 Backpropagation formulas for an MLP 13

12 Stochastic gradient descent, batches, Adam, and momentum **14**
12.1 Learning rate: follow one parameter . . . 15
12.2 Adam: two memories for every parameter . . . 15
12.3 Bias correction and the implemented update . . . 16
12.4 Read the actual training settings . . . 17

13 Training, validation, testing, and overfitting **18**

14 Normalization and nondimensionalization **18**
14.1 Statistical normalization . . . 18
14.2 Physical nondimensionalization . . . 18

15 Rarefied-flow fundamentals for Week 2 **19**
15.1 Mean free path . . . 19
15.2 Knudsen-number regimes . . . 19
15.3 Reynolds number versus Knudsen number . . . 20
15.4 Velocity slip . . . 20
15.5 Temperature jump . . . 20
15.6 Accommodation coefficient . . . 20
15.7 Knudsen layer . . . 21

16 Neural-network surrogate for a rarefied-flow quantity **21**
16.1 Why compare with baselines? . . . 21

17 Physical validation checks for Week 2 **21**
17.1 Positivity check . . . 22
17.2 Pressure-ratio monotonicity check . . . 22
17.3 Regime-wise error . . . 22
17.4 Extrapolation check . . . 22

18 How the Week-2 Colab matches this lecture **22**

19 Common student mistakes **23**

20 Study questions **23**

21 Summary **24**

1 Purpose of this lecture

Week 1 built a common starting point: fluid variables, Python arrays, TensorFlow tensors, finite differences, residuals, and the lid-driven cavity benchmark at $\mathrm{Re} = 100$. By the end of Week 1, a student should know that a velocity field can be stored as a two-dimensional array, derivatives can be approximated from neighboring grid values, and a numerical result must be validated against a benchmark or a residual.

Week 2 has a different purpose. It teaches the first real machine-learning model used in the course: a feed-forward neural network. The main goal is not to memorize software commands. The goal is to understand the model itself.

Main objective

By the end of this lecture, a neural network should no longer feel like a black box. A student should be able to explain:

- what the input features are;
- what weights and biases do;
- how one neuron computes an output;
- why activation functions are needed;
- how a multi-layer network is built from many neurons;
- what a loss function measures;
- how gradient descent and backpropagation change weights;
- why a low test error is not enough for AI in fluids;
- how neural networks become surrogates for rarefied-flow quantities.

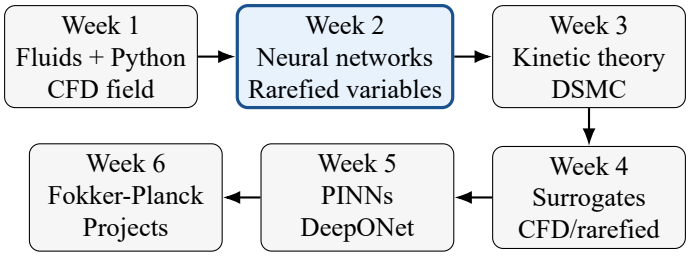
The lecture also introduces the rarefied-flow variables that will be used in the rest of the course: molecular mean free path, Knudsen number, rarefaction regimes, slip, temperature jump, accommodation coefficient, and Knudsen layers. These variables become features, targets, or validation checks in the coding lab and final projects.

What students should be able to do after Week 2

After reading this lecture and completing the lab, students should be able to:

1. construct a feature matrix X and target array Y from a fluid problem;
2. distinguish regression from classification;
3. compute the forward pass of a single neuron by hand;
4. compute one gradient-descent update for a simple one-neuron model;
5. describe the roles of weights, biases, activation functions, loss functions, learning rate, batch size, and epochs;
6. train a small neural network in Python or TensorFlow/Keras;
7. compare a neural network against a simple baseline;
8. interpret training and validation curves;
9. classify continuum, slip, transition, and free-molecular regimes using Kn;
10. explain why extrapolation across rarefaction regimes is dangerous.

How this week fits the six-week course



Week 2 is the bridge. Without it, later topics such as surrogate modeling, PINNs, DeepONet, and neural operators become vocabulary without understanding.

2 From a CFD field to a machine-learning dataset

In Week 1, a two-dimensional incompressible flow field was represented by

$$u = u(x,y), \qquad v = v(x,y), \qquad p = p(x,y). \tag{1}$$

Here x and y are spatial coordinates, u is the horizontal velocity component, v is the vertical velocity component, and p is pressure.

On a grid, Eq. (1) becomes

$$U_{j,i} \approx u(x_i,y_j), \qquad V_{j,i} \approx v(x_i,y_j), \qquad P_{j,i} \approx p(x_i,y_j). \tag{2}$$

Here i is the grid index in the x-direction, j is the grid index in the y-direction, x_i is the ith grid coordinate, y_j is the jth grid coordinate, and $U_{j,i}$, $V_{j,i}$, and $P_{j,i}$ are array entries approximating the continuous fields.

A machine-learning dataset can be built by converting each grid location, parameter value, or simulation case into an input-output pair. A generic supervised-learning dataset is

$$\mathcal{D} = \left\{ \left(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)} \right) \right\}_{i=1}^{N}. \tag{3}$$

Here $\mathcal{D}$ is the dataset, N is the number of samples, i is the sample index, $\boldsymbol{x}^{(i)} \in \mathbb{R}^{d_x}$ is the input feature vector for sample i, d_x is the number of input features, $\boldsymbol{y}^{(i)} \in \mathbb{R}^{d_y}$ is the target vector for sample i, and d_y is the number of target quantities.

Translation table

Fluid language	Computational language	ML language
Flow field $u(x,y)$	2D array $U_{j,i}$	Image-like tensor or target field
Grid point (x_i,y_j)	Array location (j,i)	Input features x_i, y_j
Physical parameter Re or Kn	Scalar value	Input feature
Boundary condition	Array edge values	Constraint or validation check
Residual	Equation mismatch	Physics diagnostic or physics loss
Benchmark data	Trusted numerical values	Test targets

The neural network does not know what a vortex, shock, or Knudsen layer means unless this information enters through features, targets, physics losses, architecture, or validation.

3 Supervised learning: vocabulary and workflow

Supervised learning means that the model sees examples of inputs and correct outputs. The model is trained to approximate the mapping from input to output.

$$\widehat{\boldsymbol{y}} = f_{\boldsymbol{\theta}}(\boldsymbol{x}). \tag{4}$$

Here $\boldsymbol{x}$ is the input feature vector, $\widehat{\boldsymbol{y}}$ is the model prediction, $f_{\boldsymbol{\theta}}$ is the machine-learning model, and $\boldsymbol{\theta}$ denotes all trainable model parameters, such as neural-network weights and biases.

Important terms

Term	Meaning in this course
Sample	One input-output example, such as one rarefied-flow case or one grid point from one simulation.
Feature	An input variable used by the model, such as $\log_{10}\mathrm{Kn}$, accommodation coefficient, pressure ratio, or spatial coordinate.
Target	The quantity the model should predict, such as mass flow rate, slip velocity, heat flux, density, temperature, or a velocity component.
Prediction	The output produced by the model before comparing with the target.
Trainable parameter	A trainable number inside the model, usually a weight or bias in a neural network.
Hyperparameter	A training choice such as learning rate, batch size or hidden-layer width; it is not updated by Adam.
Loss	A scalar number measuring prediction error on training data.
Optimizer	An algorithm that changes model parameters to reduce the loss.
Epoch	One pass through the training dataset.
Batch	A subset of training samples used for one parameter update.
Generalization	Performance on samples not used directly for parameter updates.

3.1 Regression and classification

Regression means that the target is a continuous value. Examples in fluids include predicting a mass flow rate, heat flux, velocity component, pressure coefficient, shock thickness, or slip velocity.

Classification means that the target is a discrete class. Examples include classifying a flow regime as continuum, slip, transition, or free molecular.

Task	Input features	Target
Regression: mass-flow surrogate	$[\log_{10}\mathrm{Kn}, \alpha, \Pi, \Theta]$	Q^*, normalized mass-flow proxy
Regression: pointwise field surrogate	$[\mathrm{Re}, x, y]$ or $[\mathrm{Kn}, x, y]$	u, v, T, or ρ
Classification: rarefaction regime	$[\log_{10}\mathrm{Kn}]$	continuum/slip/transition/free molecular
Classification: model choice	local Kn, gradient Kn, Ma	continuum solver / kinetic solver / hybrid solver

In the first row, α is the gas-surface accommodation coefficient, Π is a pressure ratio, Θ is a wall-temperature ratio, and Q^* is a nondimensional mass-flow proxy.

4 A first analogy: a linear decision rule

A common teaching starting point for neural networks is a two-feature decision problem. Instead of using a non-fluid example, we use a simple rarefied-flow classification example.

Suppose we want to classify a case as "continuum-like" or "rarefied" using two features:

$$x_1 = \log_{10}(\mathrm{Kn}), \qquad x_2 = \alpha. \tag{5}$$

Here x_1 is the base-10 logarithm of the Knudsen number, Kn is the Knudsen number, x_2 is the accommodation coefficient, and α measures gas-surface accommodation.

A very simple model is

$$z = w_1 x_1 + w_2 x_2 + b. \tag{6}$$

Here z is the pre-activation score, w_1 and w_2 are weights multiplying the two features, and b is the bias. The score z is then passed through a step function:

$$o = H(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases} \tag{7}$$

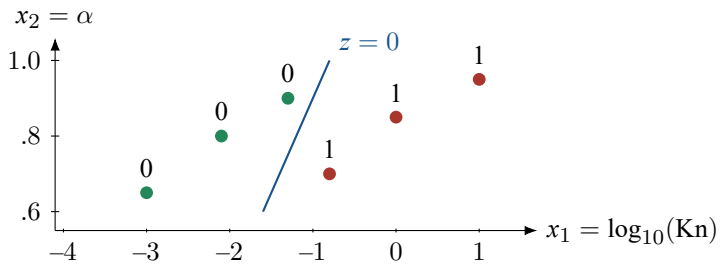
Here o is the model output and $H(z)$ is the Heaviside step activation. The output $o = 1$ could mean "rarefied" and $o = 0$ could mean "continuum-like."

The boundary between the two classes is

$$w_1 x_1 + w_2 x_2 + b = 0. \tag{8}$$

Here w_1, w_2, and b are the same parameters used in Eq. (6), and x_1 and x_2 are the features defined in Eq. (5). This equation is a line in the two-dimensional feature plane.

For this illustration choose $w_1 = 1, w_2 = -2$, and $b = 2.8$, so $z = x_1 - 2x_2 + 2.8$ and the line is $x_2 = (x_1 + 2.8)/2$.



Every sample lies in $0.6 \leq \alpha \leq 1$. For example, $z(-1.3, 0.9) = -0.3 < 0$ gives class 0, while $z(-0.8, 0.7) = 0.6 > 0$ gives class 1. These are illustrative class labels, not a physical definition of Knudsen regimes.

This is the first important idea: a neuron with a step activation draws a boundary in feature space. Learning means changing w_1, w_2, and b until that boundary separates the training examples as well as possible.

5 The artificial neuron

A single artificial neuron takes several input features, multiplies them by weights, adds a bias, and then applies an activation function.

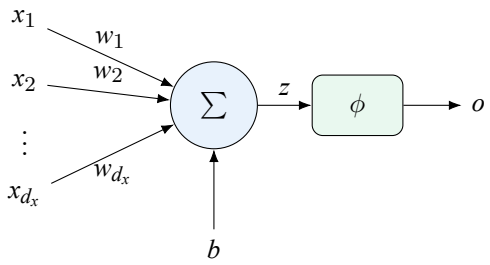
$$z = \sum_{j=1}^{d_x} w_j x_j + b = \boldsymbol{w}^T \boldsymbol{x} + b. \tag{9}$$

Here d_x is the number of input features, j is the feature index, x_j is the jth input feature, w_j is the weight multiplying x_j, b is the bias, $\boldsymbol{x} = [x_1, x_2, \ldots, x_{d_x}]^T$ is the feature vector, $\boldsymbol{w} = [w_1, w_2, \ldots, w_{d_x}]^T$ is the weight vector, T denotes transpose, and z is the pre-activation value.

The neuron output is

$$o = \phi(z). \tag{10}$$

Here o is the neuron output and ϕ is the activation function. The activation function may be a step function, sigmoid, hyperbolic tangent, ReLU, ELU, or a linear function.



5.1 Physical interpretation of weights and bias

Weights tell the model how strongly each feature affects the prediction. A large positive weight means that increasing the feature tends to increase the pre-activation value z. A large negative weight means that increasing the feature tends to decrease z. A weight close to zero means that the feature has little direct influence in that neuron.

The bias shifts the neuron threshold. Without b, the boundary $\boldsymbol{w}^T\boldsymbol{x} = 0$ must pass through the origin of feature space. With b, the boundary can move.

Fluid example

If $x_1 = \log_{10}\mathrm{Kn}$ has a positive weight in a neuron predicting rarefaction strength, increasing Kn tends to increase that neuron's output. If $x_2 = \alpha$ has a negative weight, increasing accommodation may reduce that neuron's contribution to the predicted slip correction. The final behavior of a multi-layer network is more complicated, but this single-neuron interpretation is a useful starting point.

6 Activation functions

If every neuron used only a linear operation, a deep neural network would collapse into one linear mapping. Activation functions introduce nonlinearity. Nonlinearity is essential because fluid relations are usually nonlinear: vortices, shocks, slip, temperature jump, Knudsen layers, and transition-regime effects are not linear functions of the input parameters.

Activation	Formula	Common use
Step	$\phi(z) = 0$ if $z < 0$, $\phi(z) = 1$ if $z \geq 0$	Simple perceptron classification; not convenient for gradient training because derivative is zero almost everywhere.
Sigmoid	$\phi(z) = \dfrac{1}{1+e^{-z}}$	Binary classification output or probability-like value.
Tanh	$\phi(z) = \tanh(z)$	Hidden layers when outputs centered near zero are helpful.

ReLU	$\phi(z) = \max(0, z)$	Common hidden-layer activation in deep networks.
ELU	$\phi(z) = z$ for $z \geq 0$, $\phi(z) = a(e^z - 1)$ for $z < 0$	Hidden layers; smoother negative branch than ReLU.
Linear	$\phi(z) = z$	Regression output layer when target can be any real value.

In the table, z is the pre-activation scalar defined in Eq. (9), e is the base of the natural logarithm, $a > 0$ is a chosen ELU shape parameter, and $\phi(z)$ is the activation output.

6.1 Derivatives of common activations

Backpropagation uses activation derivatives. For example,

$$\frac{\mathrm{d}}{\mathrm{d}z}\left[\frac{1}{1+e^{-z}}\right] = \sigma(z)\left[1-\sigma(z)\right], \tag{11}$$

where z is the scalar input, $\sigma(z) = 1/(1+e^{-z})$ is the sigmoid function, e is the base of the natural logarithm, and $\mathrm{d}/\mathrm{d}z$ denotes differentiation with respect to z.

For ReLU,

$$\frac{\mathrm{d}}{\mathrm{d}z}\max(0,z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases} \tag{12}$$

Here z is the scalar input. The derivative at $z = 0$ is not unique; in software it is assigned a convenient value, often zero.

7 Targets, predictions, errors, and loss functions

For one supervised-learning sample, let the target be t and the prediction be o. The prediction error is

$$e = t - o. \tag{13}$$

Here e is the scalar prediction error, t is the true target value, and o is the model output.

A common one-sample squared-error loss is

$$C = \frac{1}{2}(t-o)^2. \tag{14}$$

Here C is the cost or loss for one sample, t is the target, and o is the prediction. The factor $1/2$ is included because it cancels the factor of 2 that appears when differentiating the square.

For N samples, the mean-squared error is

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(y^{(i)} - \widehat{y}^{(i)}\right)^2. \tag{15}$$

Here N is the number of samples, i is the sample index, $y^{(i)}$ is the true scalar target for sample i, $\widehat{y}^{(i)}$ is the predicted scalar target for sample i, and MSE is the mean-squared error.

The root-mean-squared error is

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(y^{(i)} - \widehat{y}^{(i)}\right)^2}. \tag{16}$$

Here RMSE has the same units as the target quantity, while N, i, $y^{(i)}$, and $\widehat{y}^{(i)}$ are defined as in Eq. (15).

For a predicted field, a relative L_2 error is often useful:

$$\text{Rel}\, L_2 = \frac{\|\boldsymbol{y} - \widehat{\boldsymbol{y}}\|_2}{\|\boldsymbol{y}\|_2}. \tag{17}$$

Here $\boldsymbol{y}$ is the vector of true field values, $\widehat{\boldsymbol{y}}$ is the vector of predicted field values, $\|\cdot\|_2$ is the Euclidean norm, and $\text{Rel}\, L_2$ is the relative L_2 error.

Low loss is not enough

A neural-network surrogate can have a small average error but still violate physics. In fluids, always ask additional questions: Is density positive? Does the wall behavior make sense? Is mass conserved? Is the shock location correct? Is the prediction being made outside the training regime?

8 Learning: how one neuron changes its weights

Learning means changing the trainable parameters to reduce the loss. In a one-neuron model, the trainable parameters are the weights and the bias.

For a linear-output neuron,

$$o = z = \sum_{j=1}^{d_x} w_j x_j + b. \tag{18}$$

Here o is the scalar output, z is the pre-activation value, m is the number of features, x_j is the jth feature, w_j is the jth weight, and b is the bias.

Using the one-sample loss in Eq. (14), the derivative of C with respect to weight w_j is

$$\frac{\partial C}{\partial w_j} = -(t-o)x_j. \tag{19}$$

Here $\partial C/\partial w_j$ is the gradient of the loss with respect to weight w_j, C is the one-sample loss, t is the target, o is the model output, and x_j is the jth input feature.

The derivative of C with respect to the bias is

$$\frac{\partial C}{\partial b} = -(t-o). \tag{20}$$

Here $\partial C/\partial b$ is the gradient of the loss with respect to bias b, and $t-o$ is the prediction error defined in Eq. (13).

Gradient descent updates the weight using

$$w_j \leftarrow w_j - \eta \frac{\partial C}{\partial w_j}. \tag{21}$$

Here w_j is the weight being updated, $\eta > 0$ is the learning rate, C is the loss, and $\partial C/\partial w_j$ is the gradient.

The bias update is

$$b \leftarrow b - \eta \frac{\partial C}{\partial b}. \tag{22}$$

Here b is the bias, η is the learning rate, and $\partial C/\partial b$ is the gradient with respect to b.

Using Eqs. (19) and (20), the updates become

$$w_j \leftarrow w_j + \eta(t-o)x_j, \qquad b \leftarrow b + \eta(t-o). \tag{23}$$

Here w_j is the jth weight, b is the bias, η is the learning rate, t is the target, o is the prediction, and x_j is the jth feature.

8.1 Worked numerical example

Consider a one-neuron surrogate that predicts a normalized rarefied mass-flow correction Q^* from two features:

$$x_1 = \log_{10}(\mathrm{Kn}), \qquad x_2 = \alpha. \tag{24}$$

Here x_1 is the logarithmic Knudsen-number feature, Kn is the Knudsen number, x_2 is the accommodation coefficient, and α measures gas-surface accommodation.

Use the linear-output neuron

$$o = w_1 x_1 + w_2 x_2 + b. \tag{25}$$

Here o is the predicted normalized correction, w_1 and w_2 are weights, x_1 and x_2 are features, and b is the bias.

Take one training sample:

$$x_1 = -2, \qquad x_2 = 0.8, \qquad t = 0.25. \tag{26}$$

Here $x_1 = -2$ corresponds to $\mathrm{Kn} = 10^{-2}$, $x_2 = 0.8$ is the accommodation coefficient, and $t = 0.25$ is the target normalized correction.

Start with

$$w_1 = 0.20, \qquad w_2 = -0.10, \qquad b = 0.05. \tag{27}$$

Here w_1 and w_2 are initial weights and b is the initial bias.

The prediction is

$$o = (0.20)(-2) + (-0.10)(0.8) + 0.05 = -0.43. \tag{28}$$

Here o is the model output computed from Eq. (25) using the values in Eqs. (26) and (27).

The error is

$$e = t - o = 0.25 - (-0.43) = 0.68. \tag{29}$$

Here e is the prediction error, $t = 0.25$ is the target, and $o = -0.43$ is the prediction.

Choose learning rate

$$\eta = 0.05. \tag{30}$$

Here η is the learning rate controlling the update size.

Using Eq. (23), the new weights are

$$w_1^{\mathrm{new}} = 0.20 + (0.05)(0.68)(-2) = 0.132, \tag{31}$$

$$w_2^{\mathrm{new}} = -0.10 + (0.05)(0.68)(0.8) = -0.0728, \tag{32}$$

$$b^{\mathrm{new}} = 0.05 + (0.05)(0.68) = 0.084. \tag{33}$$

Here w_1^{new} and w_2^{new} are the updated weights, b^{new} is the updated bias, $\eta = 0.05$ is the learning rate, and 0.68 is the prediction error from Eq. (29).

The new prediction is

$$o^{\mathrm{new}} = (0.132)(-2) + (-0.0728)(0.8) + 0.084 = -0.23824. \tag{34}$$

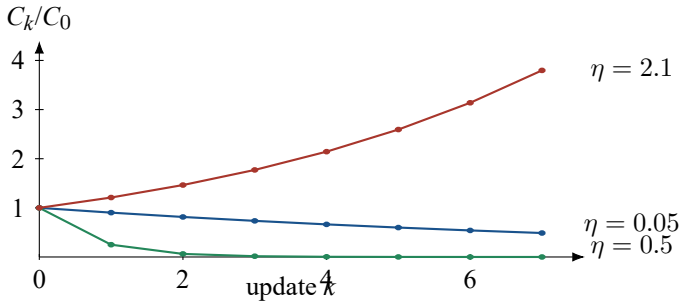
Here o^{new} is the prediction after one update. It is still far from $t = 0.25$, but it moved in the correct direction: from -0.43 to -0.23824.

This example shows what training does. Training is repeated many times over many samples until the model parameters produce acceptable predictions.

9 Learning rate, instability, and slow learning

The learning rate η in Eqs. (21) and (22) controls how large each update is.

- If η is too small, the loss decreases very slowly.
- If η is too large, the loss may oscillate or diverge.
- If η is reasonable, the loss usually decreases over training.



For the scalar loss $C(\theta) = \frac{1}{2}(\theta-3)^2$ and $\theta_0 = 1$, these are actual gradient-descent loss samples normalized by C_0. The large-rate loss grows; it is not a bounded oscillation. The notebook also shows $\eta = 1.5$, whose parameter oscillations decay. The Adam lesson below derives the error factor $1-\eta$.

10 From one neuron to a multi-layer perceptron

One neuron can only create a simple linear boundary or a simple regression formula. Real fluid relations are usually more complex. A multi-layer perceptron, or MLP, combines many neurons into layers.

Let the input layer be

$$\boldsymbol{a}^{(0)} = \boldsymbol{x}. \tag{35}$$

Here $\boldsymbol{a}^{(0)}$ is the activation vector of layer 0, and $\boldsymbol{x}$ is the input feature vector.

For layer ℓ, the pre-activation vector is

$$\boldsymbol{z}^{(\ell)} = W^{(\ell)}\boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}. \tag{36}$$

Here ℓ is the layer index, $\boldsymbol{z}^{(\ell)}$ is the pre-activation vector in layer ℓ, $W^{(\ell)}$ is the weight matrix of layer ℓ, $\boldsymbol{a}^{(\ell-1)}$ is the activation vector from the previous layer, and $\boldsymbol{b}^{(\ell)}$ is the bias vector of layer ℓ.

The activation vector is

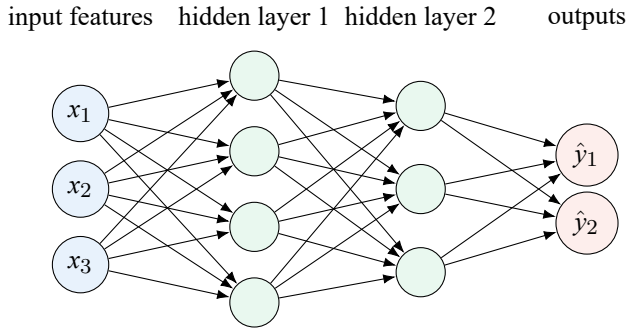
$$\boldsymbol{a}^{(\ell)} = \phi^{(\ell)}\left(\boldsymbol{z}^{(\ell)}\right). \tag{37}$$

Here $\boldsymbol{a}^{(\ell)}$ is the activation vector in layer ℓ, $\phi^{(\ell)}$ is the activation function for layer ℓ, and $\boldsymbol{z}^{(\ell)}$ is defined in Eq. (36). The activation is applied component by component.

For a network with L layers, the prediction is

$$\widehat{\boldsymbol{y}} = \boldsymbol{a}^{(L)}. \tag{38}$$

Here $\widehat{\boldsymbol{y}}$ is the model prediction, L is the index of the output layer, and $\boldsymbol{a}^{(L)}$ is the activation vector of the output layer.



The illustrated 3–4–3–2 topology has 39 weights and biases. The notebook uses a separate 4–32–32–1 topology with 1249 trainable parameters.

10.1 Why hidden layers help

Each hidden neuron creates a transformed feature. The next layer combines those transformed features. With enough neurons and nonlinear activations, an MLP can approximate complicated nonlinear functions. This is why MLPs are used for regression surrogates.

For example, a rarefied-flow surrogate may approximate

$$Q^* \approx f_{\boldsymbol{\theta}}\left(\log_{10}\mathrm{Kn}, \alpha, \Pi, \Theta\right). \tag{39}$$

Here Q^* is a nondimensional mass-flow proxy, $f_{\boldsymbol{\theta}}$ is the neural-network model, $\boldsymbol{\theta}$ denotes all trainable weights and biases, Kn is Knudsen number, α is accommodation coefficient, Π is pressure ratio, and Θ is wall-temperature ratio.

11 Backpropagation: how deep networks learn

For a one-neuron model, the gradient is easy to write directly. In a multi-layer network, a weight in an early layer affects the output only through later layers. Backpropagation is the efficient application of the chain rule to compute all these gradients.

11.1 Chain rule idea

If

$$q = f(g(s)), \tag{40}$$

where s is an input variable, g is an intermediate function, f is an outer function, and q is the final output, then

$$\frac{\mathrm{d}q}{\mathrm{d}s} = \frac{\mathrm{d}f}{\mathrm{d}g}\frac{\mathrm{d}g}{\mathrm{d}s}. \tag{41}$$

Here $\mathrm{d}q/\mathrm{d}s$ is the derivative of the output with respect to s, $\mathrm{d}f/\mathrm{d}g$ is the derivative of the outer function with respect to its input g, and $\mathrm{d}g/\mathrm{d}s$ is the derivative of the intermediate function with respect to s.

A neural network is a long composition of functions. Backpropagation applies Eq. (41) from the output layer backward to the earlier layers.

11.2 Backpropagation formulas for an MLP

Consider a network with layer equations (36) and (37). For one training sample, define the squared-error loss

$$C = \frac{1}{2} \left\| \boldsymbol{a}^{(L)} - \boldsymbol{y} \right\|_2^2. \tag{42}$$

Here C is the one-sample loss, $\boldsymbol{a}^{(L)}$ is the output-layer activation vector, L is the output-layer index, $\boldsymbol{y}$ is the target vector, and $\| \cdot \|_2$ is the Euclidean norm.

For the output layer, define the error signal

$$\boldsymbol{\delta}^{(L)} = \left(\boldsymbol{a}^{(L)} - \boldsymbol{y} \right) \odot \phi'^{(L)} \left(\boldsymbol{z}^{(L)} \right). \tag{43}$$

Here $\boldsymbol{\delta}^{(L)}$ is the output-layer error signal, $\boldsymbol{a}^{(L)}$ is the prediction vector, $\boldsymbol{y}$ is the target vector, $\odot$ denotes component-wise multiplication, $\phi'^{(L)}$ is the derivative of the output-layer activation, and $\boldsymbol{z}^{(L)}$ is the output-layer pre-activation vector.

For a hidden layer ℓ, the error signal is

$$\boldsymbol{\delta}^{(\ell)} = \left[(W^{(\ell+1)})^T \boldsymbol{\delta}^{(\ell+1)} \right] \odot \phi'^{(\ell)} \left(\boldsymbol{z}^{(\ell)} \right). \tag{44}$$

Here $\boldsymbol{\delta}^{(\ell)}$ is the error signal in hidden layer ℓ, $W^{(\ell+1)}$ is the weight matrix of the next layer, T denotes transpose, $\boldsymbol{\delta}^{(\ell+1)}$ is the next-layer error signal, $\phi'^{(\ell)}$ is the derivative of the activation in layer ℓ, and $\boldsymbol{z}^{(\ell)}$ is the pre-activation vector in layer ℓ.

The gradients are

$$\frac{\partial C}{\partial W^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} \left(\boldsymbol{a}^{(\ell-1)} \right)^T, \tag{45}$$

where $\partial C / \partial W^{(\ell)}$ is the gradient matrix for weights in layer ℓ, $\boldsymbol{\delta}^{(\ell)}$ is the layer-ℓ error signal, and $\boldsymbol{a}^{(\ell-1)}$ is the activation vector from the previous layer.

The bias gradient is

$$\frac{\partial C}{\partial \boldsymbol{b}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)}. \tag{46}$$

Here $\partial C / \partial \boldsymbol{b}^{(\ell)}$ is the gradient of the loss with respect to the bias vector in layer ℓ, and $\boldsymbol{\delta}^{(\ell)}$ is the layer error signal.

The gradient-descent updates are

$$W^{(\ell)} \leftarrow W^{(\ell)} - \eta \frac{\partial C}{\partial W^{(\ell)}}, \tag{47}$$

$$\boldsymbol{b}^{(\ell)} \leftarrow \boldsymbol{b}^{(\ell)} - \eta \frac{\partial C}{\partial \boldsymbol{b}^{(\ell)}}. \tag{48}$$

Here $W^{(\ell)}$ and $\boldsymbol{b}^{(\ell)}$ are the trainable weights and biases in layer ℓ, η is the learning rate, and the gradients are defined in Eqs. (45) and (46).

Backpropagation in words

Backpropagation asks: if the output is wrong, how much should each weight and bias in every layer be blamed? The algorithm computes this blame using derivatives, then the optimizer adjusts the parameters to reduce future error.

12 Stochastic gradient descent, batches, Adam, and momentum

In real training, we usually do not compute gradients using the entire dataset at every update. Instead, we use mini-batches.

For a mini-batch $\mathcal{B}$, the batch loss is

$$C_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} C^{(i)}. \tag{49}$$

Here $\mathcal{B}$ is a set of sample indices in the mini-batch, $|\mathcal{B}|$ is the number of samples in the mini-batch, i is a sample index, $C^{(i)}$ is the loss for sample i, and $C_{\mathcal{B}}$ is the average mini-batch loss.

Stochastic gradient descent updates a parameter vector $\boldsymbol{\theta}$ as

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla_{\boldsymbol{\theta}} C_{\mathcal{B}}(\boldsymbol{\theta}_k). \tag{50}$$

Here $\boldsymbol{\theta}_k$ is the parameter vector at update step k, $\boldsymbol{\theta}_{k+1}$ is the parameter vector after the update, η is the learning rate, $\nabla_{\boldsymbol{\theta}}$ denotes the gradient with respect to $\boldsymbol{\theta}$, and $C_{\mathcal{B}}$ is the mini-batch loss.

Momentum smooths the update direction:

$$\boldsymbol{v}_{k+1} = \beta \boldsymbol{v}_k + (1-\beta)\boldsymbol{g}_k, \tag{51}$$

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \boldsymbol{v}_{k+1}. \tag{52}$$

Here $\boldsymbol{v}_k$ is the momentum vector at step k, $\beta \in [0, 1)$ is the momentum coefficient, $\boldsymbol{g}_k = \nabla_{\boldsymbol{\theta}} C_{\mathcal{B}}(\boldsymbol{\theta}_k)$ is the current gradient, η is the learning rate, and $\boldsymbol{\theta}_k$ is the parameter vector.

12.1 Learning rate: follow one parameter

A parameter is a trainable weight or bias. The vector θ collects all of them. A batch is a subset of training samples; its loss C_B is their average prediction error. The gradient points locally uphill in parameter space, so an optimizer takes a step in the opposite direction. The update index k counts parameter changes, not epochs and not physical CFD time steps.

$$g_k = \nabla_\theta C_B(\theta_{k-1}), \qquad \theta_k = \theta_{k-1} - \eta g_k.$$

The learning rate $\eta > 0$ sets the step scale. For one scalar parameter, take

$$C(\theta) = \tfrac{1}{2}(\theta - 3)^2, \qquad g = \frac{\mathrm{d}C}{\mathrm{d}\theta} = \theta - 3.$$

Starting from $\theta_0 = 1$ gives $g_1 = -2$. With $\eta = 0.5$, the next parameter is $\theta_1 = 1 - 0.5(-2) = 2$: the negative gradient moves it toward the minimum at $\theta = 3$. Repeating the update gives

$$\underbrace{\theta_k - 3}_{e_k} = (1 - \eta)\,\underbrace{(\theta_{k-1} - 3)}_{e_{k-1}}.$$

Thus this quadratic gradient-descent example converges when $|1-\eta| < 1$, or $0 < \eta < 2$. For $\eta = 0.05$ it approaches slowly; for $\eta = 0.5$ its error halves each update; for $\eta = 2.1$ its error changes sign and grows. This bound depends on this loss's curvature. It is not a general neural-network or Adam learning-rate limit.

Notebook experiment. Plot the parameter and the loss against update number for these rates, including $\eta = 1.5$ (alternating, decaying errors). Keep the starting value and loss fixed so that the effect of the rate can be identified. The experiment uses NumPy and does not retrain the fluid surrogate.

12.2 Adam: two memories for every parameter

Adam stores a running average m_k of signed gradients and a running average v_k of squared gradients. Both arrays have the same shape as θ. The symbol v_k in this optimizer is memory, not a fluid velocity. Starting with $m_0 = v_0 = 0$,

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k,$$
$$v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^2.$$

The parameters $0 \le \beta_1, \beta_2 < 1$ control memory. The notebook uses the Keras defaults $\beta_1 = 0.9$ and $\beta_2 = 0.999$. Multiplication, squaring, division and square roots operate separately on every component. The first memory smooths the direction; the second measures recent gradient magnitude. Dividing by a square-root magnitude makes the step scale adaptive for each parameter. The second memory averages squared gradients; it is not a statistical variance. It does not remove the base learning-rate choice.

12.3 Bias correction and the implemented update

The zero initial memories underestimate the early average magnitudes. After k updates their bias-corrected versions are

$$\widehat{m}_k = \frac{m_k}{1-\beta_1^k}, \qquad \widehat{v}_k = \frac{v_k}{1-\beta_2^k}.$$

The familiar Adam algorithm writes

$$\theta_k = \theta_{k-1} - \eta \frac{\widehat{m}_k}{\sqrt{\widehat{v}_k} + \epsilon_{\text{alg}}}.$$

Here the small positive ϵ_{alg} prevents division by zero. For exact reproduction, Keras uses the paper's *epsilon-hat* convention:

$$\alpha_k = \eta \frac{\sqrt{1-\beta_2^k}}{1-\beta_1^k}, \qquad \boxed{\theta_k = \theta_{k-1} - \alpha_k \frac{m_k}{\sqrt{v_k} + \epsilon_K}.}$$

Its default is $\epsilon_K = 10^{-7}$. The equivalent denominator in the bias-corrected form contains $\epsilon_K/\sqrt{1-\beta_2^k}$; the two epsilon symbols should not be assigned the same value when reproducing an exact step.

One numerical Adam step. For the preceding quadratic, $\theta_0 = 1$ and $g_1 = -2$. Hence

$$m_1 = 0.1(-2) = -0.2, \qquad v_1 = 0.001(4) = 0.004,$$

$$\widehat{m}_1 = -0.2/0.1 = -2, \qquad \widehat{v}_1 = 0.004/0.001 = 4.$$

Use $\eta = 0.1$ only for this visible arithmetic example. Substitution into the Keras-form update gives $\theta_1 \simeq 1.09999984$. The notebook executes this calculation and checks its value. The fluid surrogate keeps its separate learning rate of 0.001.

Using the same $\eta = 0.1$, gradient descent would move by 0.2, giving $\theta_1 = 1.2$, while Adam moves by approximately 0.1. With uncorrected memories, the alternative update $-\eta m_1/(\sqrt{v_1} + \epsilon_K)$ would move by approximately 0.316. Bias correction changes the early step scale.

12.4 Read the actual training settings

The 5000 samples split into 3500 training, 750 validation and 750 test samples. The 4–32–32–1 network has $4(32)+32+32(32)+32+32(1)+1=1249$ trainable parameters. Each epoch contains 55 updates: 54 batches of 64 samples and a final batch of 44. Running all 500 epochs would give 27,500 updates.

Setting	Meaning in the Week 2 training call
`learning_rate=1e-3`	Base Adam step scale $\eta = 0.001$.
`batch_size=64`	Up to 64 samples contribute to each gradient update.
One epoch	One pass through the training set: approximately $\lceil N_{\text{train}}/64 \rceil$ updates, including a smaller final batch.
`epochs=500`	Maximum number of passes, not a promise to run all 500.
`patience=30`	Stop after 30 epochs without lower validation loss (default improvement threshold: zero).
`restore_best_weights=True`	Restore weights from the best monitored epoch.

If epoch 120 has the lowest validation loss and the next 30 epochs do not improve it, training ends after epoch 150 and restores epoch-120 weights. Early stopping does not reduce the learning rate. For each independent rate trial, call `keras.utils.set_random_seed(7)` before rebuilding the model, optimizer and callback. Hold the split, training-only scaling, architecture and batch policy fixed; compare matched seeds. Select using validation loss; reserve test data for final evaluation. Training loss alone does not establish physical accuracy or extrapolation.

Sources. Kingma and Ba, *Adam: A Method for Stochastic Optimization* (ICLR 2015), `https://arxiv.org/abs/1412.6980`; Keras Adam documentation, `https://keras.io/api/optimizers/adam/`.

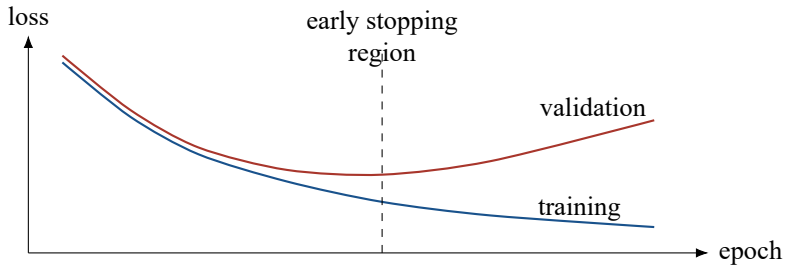
13 Training, validation, testing, and overfitting

The data are usually split into training, validation, and test sets:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}. \tag{53}$$

Here $\mathcal{D}$ is the full dataset, $\mathcal{D}_{\text{train}}$ is the training set used to update model parameters, $\mathcal{D}_{\text{val}}$ is the validation set used to tune modeling choices, and $\mathcal{D}_{\text{test}}$ is the test set used for final evaluation.

Overfitting occurs when the model performs well on training data but poorly on unseen data. In fluids, overfitting is especially dangerous because the model may memorize a few simulation cases while failing in a new physical regime.



An epoch is one complete pass through the training data; the horizontal axis counts completed epochs. Training loss often decreases continuously, but validation loss can increase after overfitting begins.

14 Normalization and nondimensionalization

Neural networks train better when inputs and targets have reasonable scales. In fluids, scaling has two meanings: statistical normalization and physical nondimensionalization.

14.1 Statistical normalization

Standardization of a scalar feature is

$$x_{\text{std}} = \frac{x - \mu_x}{\sigma_x}. \tag{54}$$

Here x is the original scalar feature, μ_x is the mean of that feature in the training set, σ_x is the standard deviation of that feature in the training set, and x_{std} is the standardized feature.

Min-max scaling is

$$x_{\text{mm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}. \tag{55}$$

Here x is the original scalar feature, $x_{\min}$ is the minimum training-set value, $x_{\max}$ is the maximum training-set value, and x_{mm} is the min-max scaled feature.

14.2 Physical nondimensionalization

The Reynolds number is

$$\text{Re} = \frac{\rho U L_c}{\mu} = \frac{U L_c}{\nu}. \tag{56}$$

Here Re is the Reynolds number, ρ is density, U is a characteristic velocity, L_c is a characteristic length, μ is dynamic viscosity, and $\nu = \mu/\rho$ is kinematic viscosity.

The Knudsen number is

$$\text{Kn} = \frac{\lambda}{L_c}. \tag{57}$$

Here Kn is the Knudsen number, λ is the molecular mean free path, and L_c is a characteristic macroscopic length.

The Mach number is

$$\mathrm{Ma} = \frac{U}{a}. \tag{58}$$

Here Ma is the Mach number, U is a characteristic flow velocity, and a is the speed of sound.

Practical rule for rarefied-flow ML

Use physical nondimensional groups first, then apply statistical normalization to those groups. For rarefied-flow datasets, use $\log_{10}\mathrm{Kn}$ more often than Kn, because Kn may span several orders of magnitude.

15 Rarefied-flow fundamentals for Week 2

This course is not only about generic neural networks. It is about neural networks for fluid dynamics, especially rarefied and non-equilibrium gas flows. Therefore, students need the main rarefied-flow variables before building a surrogate.

15.1 Mean free path

The molecular mean free path is the average distance traveled by a molecule between collisions. A common hard-sphere estimate is

$$\lambda = \frac{k_B T}{\sqrt{2}\pi d^2 p}. \tag{59}$$

Here λ is the mean free path, k_B is Boltzmann's constant, T is gas temperature, d is the molecular diameter, and p is gas pressure.

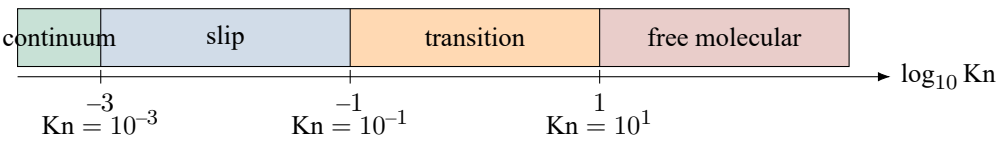
Equation (59) shows that lower pressure increases λ. Small devices also increase Kn because L_c becomes small in Eq. (57).

15.2 Knudsen-number regimes

A common regime classification is

Regime	Approximate range	Modeling implication
Continuum	$\mathrm{Kn} < 10^{-3}$	Navier-Stokes with no-slip walls is often appropriate.
Slip	$10^{-3} \le \mathrm{Kn} < 10^{-1}$	Continuum equations may work with velocity-slip and temperature-jump boundary conditions.
Transition	$10^{-1} \le \mathrm{Kn} < 10$	Strong non-equilibrium effects; DSMC, Boltzmann-type, BGK, or Fokker-Planck methods are often needed.
Free molecular	$\mathrm{Kn} \ge 10$	Molecule-wall interactions dominate over molecule-molecule collisions.

Here Kn is the Knudsen number defined in Eq. (57). The boundaries between regimes are approximate; real model validity depends on geometry, gradients, Mach number, and wall physics.



15.3 Reynolds number versus Knudsen number

The Reynolds number in Eq. (56) compares inertia to viscous diffusion. The Knudsen number in Eq. (57) compares molecular length scale to device length scale. They answer different questions.

Number	Measures	Main question
Re	inertia relative to viscosity	Is the flow dominated by inertia, viscosity, instability, or turbulence?
Kn	molecular mean free path relative to geometry	Is the continuum assumption valid?
Ma	flow speed relative to sound speed	Is compressibility important?

15.4 Velocity slip

At a no-slip continuum wall,

$$u_{\text{gas}}|_w = u_w. \tag{60}$$

Here $u_{\text{gas}}|_w$ is the gas tangential velocity evaluated at the wall, and u_w is the wall tangential velocity.

In rarefied flow, a simplified first-order slip relation is

$$u_{\text{slip}} = u_{\text{gas}}|_w - u_w \approx C_u \lambda \left.\frac{\partial u_t}{\partial n}\right|_w. \tag{61}$$

Here u_{slip} is the slip velocity, $u_{\text{gas}}|_w$ is gas tangential velocity at the wall, u_w is wall tangential velocity, C_u is a slip coefficient depending on gas-surface interaction, λ is mean free path, u_t is tangential gas velocity, n is the wall-normal coordinate, and $|_w$ indicates evaluation at the wall.

15.5 Temperature jump

At a continuum isothermal wall,

$$T_{\text{gas}}|_w = T_w. \tag{62}$$

Here $T_{\text{gas}}|_w$ is the gas temperature at the wall and T_w is the wall temperature.

In rarefied flow, a simplified temperature-jump relation is

$$T_{\text{gas}}|_w - T_w \approx C_T \lambda \left.\frac{\partial T}{\partial n}\right|_w. \tag{63}$$

Here $T_{\text{gas}}|_w - T_w$ is the temperature jump, C_T is a temperature-jump coefficient, λ is mean free path, T is gas temperature, n is the wall-normal coordinate, and $|_w$ indicates wall evaluation.

15.6 Accommodation coefficient

The accommodation coefficient α measures how strongly gas molecules equilibrate with a surface. In simplified terms:

- $\alpha \approx 1$ means strong accommodation, often associated with diffuse-like reflection;
- $\alpha < 1$ means partial accommodation and more memory of incoming molecular velocity.

Accommodation affects slip, temperature jump, and energy exchange at walls. Therefore, it is a natural feature in rarefied-flow machine-learning datasets.

15.7 Knudsen layer

A Knudsen layer is a near-wall non-equilibrium region with thickness on the order of a few mean free paths. Inside this layer, continuum constitutive laws may not accurately describe stress and heat flux.

For ML, the important message is: a model can have low global error but still fail near the wall. Therefore, wall error and near-wall validation are important.

16 Neural-network surrogate for a rarefied-flow quantity

The Week-2 coding lab uses a synthetic rarefied-flow relation. It is not a replacement for DSMC or experiments. It is a controlled teaching problem where the true function is known.

The input feature vector is

$$\boldsymbol{x} = \begin{bmatrix} r \\ \alpha \\ \Pi \\ \Theta \end{bmatrix}, \qquad r = \log_{10} \mathrm{Kn}, \qquad \Pi = \frac{p_{\mathrm{in}}}{p_{\mathrm{out}}}, \qquad \Theta = \frac{T_w}{T_0}. \tag{64}$$

Here $\boldsymbol{x}$ is the feature vector, r is the logarithmic Knudsen-number feature, Kn is Knudsen number, α is accommodation coefficient, Π is inlet-to-outlet pressure ratio, p_{in} is inlet pressure, p_{out} is outlet pressure, Θ is wall-to-reference temperature ratio, T_w is wall temperature, and T_0 is reference temperature.

A synthetic normalized mass-flow proxy is

$$Q^* = \left(1 - \frac{1}{\Pi}\right) \left[1 + C_s \frac{2-\alpha}{\alpha} \frac{\mathrm{Kn}}{1+\mathrm{Kn}}\right] \Theta^{-1/2} \left[1 + C_r \tanh\left(\log_{10} \mathrm{Kn} + 1\right)\right] + \varepsilon. \tag{65}$$

Here Q^* is the synthetic nondimensional mass-flow proxy, Π is pressure ratio, C_s is a chosen slip-strength coefficient, α is accommodation coefficient, Kn is Knudsen number, Θ is temperature ratio, C_r is a chosen rarefaction-transition coefficient, tanh is the hyperbolic tangent function, and ε is a small noise term representing numerical or measurement noise.

The neural network approximates

$$\widehat{Q}^* = f_{\boldsymbol{\theta}}(r, \alpha, \Pi, \Theta). \tag{66}$$

Here $\widehat{Q}^*$ is the predicted normalized mass-flow proxy, $f_{\boldsymbol{\theta}}$ is the neural-network model, $\boldsymbol{\theta}$ denotes all weights and biases, and r, α, Π, and Θ are features defined in Eq. (64).

16.1 Why compare with baselines?

A neural network should not be accepted just because it is nonlinear. It should be compared with simple baselines:

- a constant mean predictor;
- polynomial regression;
- interpolation or nearest-neighbor prediction;
- a simple physical scaling relation.

If a DNN does not outperform these baselines, the DNN is not useful for that task.

17 Physical validation checks for Week 2

For the synthetic Q^* model in Eq. (65), useful checks include:

17.1 Positivity check

A mass-flow proxy should not be negative in the selected parameter range:

$$\widehat{Q}^* \geq 0. \tag{67}$$

Here $\widehat{Q}^*$ is the DNN-predicted normalized mass-flow proxy.

17.2 Pressure-ratio monotonicity check

For fixed Kn, α, and Θ, increasing Π should increase the driving pressure effect in Eq. (65). A discrete check is

$$\widehat{Q}^*(\Pi_2) - \widehat{Q}^*(\Pi_1) \geq 0 \quad \text{for} \quad \Pi_2 > \Pi_1. \tag{68}$$

Here $\widehat{Q}^*(\Pi_1)$ and $\widehat{Q}^*(\Pi_2)$ are DNN predictions at pressure ratios Π_1 and Π_2, with all other features fixed.

17.3 Regime-wise error

Compute separate errors in each Knudsen regime:

$$\mathrm{RMSE}_{\mathcal{R}} = \sqrt{\frac{1}{N_{\mathcal{R}}} \sum_{i \in \mathcal{R}} \left(Q^{*(i)} - \widehat{Q}^{*(i)} \right)^2}. \tag{69}$$

Here $\mathrm{RMSE}_{\mathcal{R}}$ is the root-mean-squared error in regime $\mathcal{R}$, $N_{\mathcal{R}}$ is the number of samples in that regime, i is a sample index in the regime, $Q^{*(i)}$ is the true synthetic value, and $\widehat{Q}^{*(i)}$ is the predicted value.

A model can have acceptable total RMSE while failing badly in the transition regime. Regime-wise reporting is therefore essential.

17.4 Extrapolation check

If training uses only

$$-3 \leq \log_{10} \mathrm{Kn} \leq -1, \tag{70}$$

where Kn is Knudsen number, then predictions at $\log_{10} \mathrm{Kn} > -1$ are extrapolations into transition or free-molecular regimes. A DNN may return smooth outputs there, but smoothness is not physical validity.

18 How the Week-2 Colab matches this lecture

The Colab notebook follows this sequence:

1. build one-neuron predictions by hand;
2. plot activation functions;
3. demonstrate one gradient-descent update;
4. generate the synthetic rarefied-flow dataset from Eq. (65);
5. classify samples by Kn regime;
6. split data into training, validation, and test sets;
7. train polynomial-regression and nearest-neighbor baselines;
8. train a small DNN;
9. compare errors inside and outside the training range;
10. perform physical validation checks.

Listing 1: Typical Keras model for the Week-2 lab.

```
model = keras.Sequential([
    layers.Input(shape=(4,)),
    layers.Dense(32, activation="tanh"),
    layers.Dense(32, activation="tanh"),
    layers.Dense(1, activation="linear")
])
model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3),
              loss="mse",
              metrics=["mae"])
```

In this code, the input dimension is 4 because the features are $r = \log_{10}\mathrm{Kn}$, α, Π, and Θ. The two hidden layers each contain 32 neurons. The output layer has one neuron because the target is the scalar Q^*. The linear output activation is used because this is a regression problem.

19 Common student mistakes

Mistake 1: Treating the DNN as magic

A DNN is a parameterized function. It maps inputs to outputs. It does not know conservation laws, wall physics, or regime boundaries unless those are represented in the data, features, loss, architecture, or validation.

Mistake 2: Training and testing on the same data

If the same samples are used for training and testing, the reported error is not a meaningful measure of generalization.

Mistake 3: Scaling the test set using test statistics

Scalers must be fit on the training set only. Then the same scaling transformation is applied to validation and test data.

Mistake 4: Ignoring extrapolation

A random split may hide extrapolation problems. For rarefied flows, deliberately test outside the training Kn range to see whether the model is robust.

Mistake 5: Reporting only one number

A single RMSE value is not enough. Report baseline comparisons, regime-wise errors, physical checks, and limitations.

20 Study questions

Students should be able to answer the following without looking at code:

1. What are the features and target in the Week-2 rarefied-flow surrogate?
2. In $z = \boldsymbol{w}^T\boldsymbol{x} + b$, what are z, $\boldsymbol{w}$, $\boldsymbol{x}$, and b?
3. Why is a bias needed in a neuron?
4. Why does a neural network need activation functions?

5. What is the difference between regression and classification?
6. What does the learning rate control?
7. What does backpropagation compute?
8. Why can training loss decrease while validation loss increases?
9. Why is $\log_{10}$ Kn often a better feature than Kn?
10. Why is extrapolation from the slip regime to the transition regime risky?
11. What two physical checks would you apply to a rarefied mass-flow surrogate?
12. Why should every DNN be compared against a simpler baseline?

21 Summary

A neural network is a trainable nonlinear function. It is built from neurons, and each neuron computes a weighted sum, adds a bias, and passes the result through an activation function. Learning means changing the weights and biases to reduce a loss function. Backpropagation computes the gradients needed for this learning process.

For AI in fluids, the network is only one part of the workflow. The full workflow includes physical nondimensionalization, feature design, train/validation/test splitting, baseline comparison, numerical error metrics, and physical validation. In rarefied flows, the most important physical feature is often the Knudsen number, because it indicates when continuum assumptions weaken and kinetic effects become important.

Final message of Week 2

A useful neural-network surrogate is not the one with the lowest training loss. A useful surrogate is the one that predicts accurately, beats simple baselines, respects known physical behavior, and clearly states where it is likely to fail.

References and supporting materials

1. J. Nathan Kutz, *Data-Driven Modeling & Scientific Computation: Methods for Integrating Dynamics of Complex Systems and Big Data*, second-edition course manuscript.
2. E. Roohi, H. Akhlaghi, and S. Stefanov, *Advances in Direct Simulation Monte Carlo: From Micro-Scale to Rarefied Flow Phenomena*, Springer Nature, 2025.
3. G. A. Bird, *Molecular Gas Dynamics and the Direct Simulation of Gas Flows*, Oxford University Press, 1994.
4. C. Cercignani, *Rarefied Gas Dynamics: From Basic Concepts to Actual Calculations*, Cambridge University Press, 2000.
5. S. Mirjalili, neural-network lecture set used as supporting teaching material: analogy, neuron model, terminology, learning in neural networks, cost functions, activation functions, multi-layer perceptron, deep neural networks, backpropagation, momentum, and regression using MLPs.